Distributed Lupus in Fabula

Final Report for the Distributed Systems Course 2025/2026

Jacopo Bonifazi
author1@email.it

Alexandru Nicolescu
author2@gmail.com

Leonardo Vorabbi
leonardo.vorabbi@studio.unibo.it

July 15, 2026

Distributed Lupus in Fabula is a real-time multiplayer web application that brings the classic social deduction game Lupus in Fabula online, allowing groups of players to create or join a game lobby from any browser and play a full match together over the internet. The system is composed of a Vue 3 single-page frontend and a FastAPI/Socket.IO backend, communicating via REST for stateless operations and via WebSocket for real-time, low-latency interactions such as votes, phase transitions, and private role actions. Game and lobby state is kept in Redis rather than a traditional database, acting both as a shared source of truth across backend replicas and as a Pub/Sub broker that lets events generated on one instance reach clients connected to any other instance. This design allows the backend to run as multiple concurrent, interchangeable replicas behind a reverse proxy, supporting many simultaneous matches while keeping every player's view of the game synchronized in real time.

Disclaimer

During the preparation of this work, the authors used LLMs to to refine syntax and correct linguistic errors.

After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the final report/artifact

1 Concept

1.1 Type of Product

The project is a web application, specifically a real-time multiplayer game inspired by the traditional social deduction game *Lupus in Fabula*. It is a full-stack distributed system composed of:

- A **frontend**: a Vue 3 Single Page Application (SPA), accessible via browser.
- A **backend**: a set of stateless, horizontally replicated web services built with FastAPI and Socket.IO, responsible for game logic, session management, and real-time event propagation.
- A **shared data layer**: Redis, used both as a low-latency state store and as a Pub/Sub message broker connecting the backend replicas.

The whole system is designed to be deployed as a multi-container distributed application, rather than as a single monolithic server.

1.2 Use Case Description

What the software does. The application lets a group of players create or join a virtual "lobby," wait until everyone is ready, and then play a phase-based game of hidden roles and social deduction. Once a match starts, the game state moves automatically through a fixed sequence of phases:

$$\textbf{LOBBY} \rightarrow \textbf{DAY} \rightarrow \textbf{VOTING} \rightarrow \textbf{NIGHT} \rightarrow \cdots \rightarrow \textbf{ENDED}$$

driven by server-side timers.

During DAY/VOTING, players discuss and vote to eliminate a suspect; during NIGHT, players with special roles perform hidden actions that are resolved by the server before the next day begins. The match ends automatically once a win condition is met.

Where the users are. Players are geographically dispersed (or at least not necessarily co-located): each player connects independently from their own browser, over the public Internet, to a shared game session identified by a room/lobby code.

When and how frequently they interact. Interaction is continuous and real-time for the whole duration of a match (several minutes to tens of minutes), not just at fixed request/response intervals. Players must react within phase time limits (e.g., voting windows, night-action windows), so the system needs low-latency, event-driven communication rather than simple periodic polling.

How they interact. All interaction happens through a web browser via the Vue SPA. The frontend communicates with the backend in two complementary ways:

- **REST** calls for stateless operations such as creating a lobby, joining a lobby, or fetching the current game state.
- **WebSocket** for real-time, bidirectional events: votes, phase changes, chat, private role actions, disconnect/reconnect notifications, etc.

Does the system store user data? Which, and where? The system does not use a persistent relational database for user accounts since there is no long-term profile storage. Instead, it relies on Redis as a shared, TTL-based store for all session-scoped game data:

- room/lobby state and configuration;
- the list of connected players and their ready status;
- assigned roles (Wolf, Seer, Villager);
- votes cast during the DAY/VOTING phase and private night actions;
- phase timers and current phase;
- host identity.

On the client side, the browser itself retains a minimal amount of session data in local storage, purely to support reconnection and continuity of a player's session across page reloads or brief disconnects.

Roles. The system distinguishes several actor roles at different levels:

- *Application-level roles*: **Villager**, **Wolf**, **Seer**, each with different permitted actions and visibility (e.g., Wolves share a private night chat, the Seer can inspect one player per night).
- *Session-level roles*: the **Host**, who configures the match, can remove players from the lobby, and starts the game; versus **regular players**, who can only join, ready up, vote, and act according to their assigned role.
- *System role*: the backend itself acts as an autonomous **orchestrator**. It owns the authoritative game state, advances phases via timers, resolves votes and night actions, and enforces which actions are legal at any given moment.

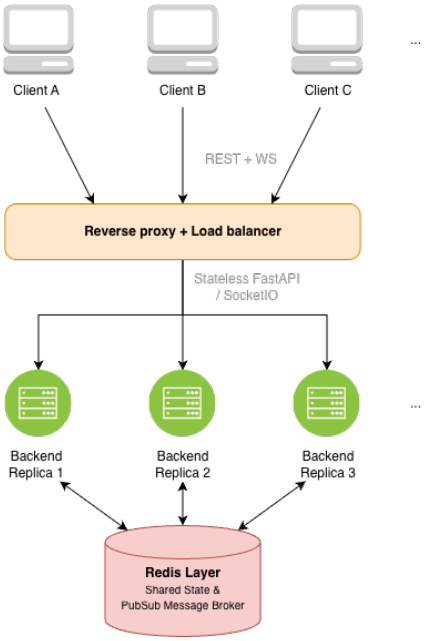

Figure 1: High-level architecture: clients connect through a reverse proxy to stateless backend replicas, which share game state and propagate events through Redis.

1.3 Why Distribution Is Needed

Distribution in this project is not motivated by geographic proximity to users, but primarily by scalability, real-time synchronization, and fault tolerance:

- **Horizontal scalability / resource sharing.** The backend is designed to run as multiple stateless replicas behind a reverse proxy, so that many concurrent lobbies and matches can be served in parallel without a single process becoming a bottleneck. Because game state lives in Redis rather than in each process's memory, any replica can handle requests for any room, allowing the system to scale out simply by adding more backend instances.
- **Shared state across replicas.** Since each backend instance is stateless with respect to game data, Redis acts as the single source of truth for lobby/game state. This is what makes horizontal scaling actually work: a player's WebSocket connection can be handled by one replica while another replica processes a REST call for the same match, and both see a consistent view of the state.
- **Cross-replica real-time propagation.** Because clients belonging to the same room can be connected to different backend replicas, a simple in-process event emitter is not sufficient. The system uses Redis Pub/Sub to broadcast game events (vote updates, phase transitions, chat messages, etc.) across all replicas, ensuring that every client in a room receives updates regardless of which instance generated them or which instance is handling their socket.

- **Fault tolerance and continuity.** The backend includes explicit mechanisms for reconnecting to Redis, retrying failed operations, sweeping/recovering timers, and reconciling state after temporary inconsistencies (anti-entropy). This matters because a real-time game cannot simply crash or stall if a single backend replica fails, a Redis connection drops momentarily, or a player's browser disconnects and reconnects mid-match. The match state and timers must survive these events.
- **Reduced dependency on a single instance.** Because the game state is decoupled from any specific process, a backend replica can be restarted, replaced, or scaled down/up without necessarily terminating ongoing matches, and a reconnecting player can resume their session against a different replica than the one they were originally connected to.

In summary, this system is distributed primarily to support many concurrent matches, keep real-time state synchronized across independent backend replicas, and remain resilient to instance failures and client disconnections.

2 Requirements Elicitation and Analysis

This section specifies *what* the system must do, independently of the specific implementation strategies later adopted. Requirements are grouped into functional, non-functional, and implementation requirements, each with a unique identifier and an associated acceptance criterion to be used during validation.

2.1 Functional Requirements

FR1 – Lobby creation. The system shall allow a user to create a new game lobby by providing a valid nickname.
Acceptance criterion: given a valid nickname, the system creates a room, assigns it a lobby code, and redirects the user to the lobby screen.

FR2 – Joining an existing lobby. The system shall allow a user to join an existing lobby using a room code and a nickname.
Acceptance criterion: with a valid room code, the player enters the lobby and appears in the participant list; with an invalid code, the system displays an error.

FR3 – Browsing open lobbies. The system shall display to the user the list of currently available lobbies.
Acceptance criterion: the home screen shows only lobbies that are still joinable, periodically refreshed from the backend.

FR4 – Host management. The system shall designate a host for each lobby and restrict room-control operations to that host.
Acceptance criterion: only the host can modify lobby settings, remove a player, and start the match.

FR5 – Ready status. The system shall allow players to declare themselves ready or not ready within the lobby.
Acceptance criterion: each player's ready status is updated accordingly, and the match can only start once the minimum required conditions are met.

FR6 – Match start and role assignment. The system shall start a match when requested by the host and assign a role to each participant.
Acceptance criterion: at match start, every player receives a role and the match enters the expected phase sequence.

FR7 – Game phase management. The system shall support the phases LOBBY, DAY, VOTING, NIGHT, and ENDED.
Acceptance criterion: the system displays the current phase and automatically transitions to the next phase according to the game rules.

FR8 – Voting and special actions. The system shall allow players to vote during the day and allow special roles to perform night actions.
Acceptance criterion: votes and actions are accepted only during the correct phase and produce an update visible to the other clients.

FR9 – In-game chat. The system shall allow participants in a room to exchange messages.
Acceptance criterion: a message sent by a user appears to the other players of the same match without a page refresh.

FR10 – Reconnection and session recovery. The system shall allow a disconnected player to reconnect to the same match.
Acceptance criterion: if the client rejoins with the same identity and the same room code, it recovers the correct current match state.

FR11 – Match end and result. The system shall determine the winner and display the final results screen.
Acceptance criterion: when a victory condition is met, the system transitions to ENDED and displays the winning faction, the roles, and the final status of all players.

2.2 Non-Functional Requirements

NFR1 – State consistency. The state of a match shall be consistent across all clients and across all backend replicas.
Acceptance criterion: two clients connected to the same room observe the same match state and the same host, even when served by different backend replicas.

NFR2 – Availability. The system shall remain usable even in the presence of a backend replica failure.
Acceptance criterion: if a backend replica goes down, clients can continue interacting with the match through the remaining active replicas.

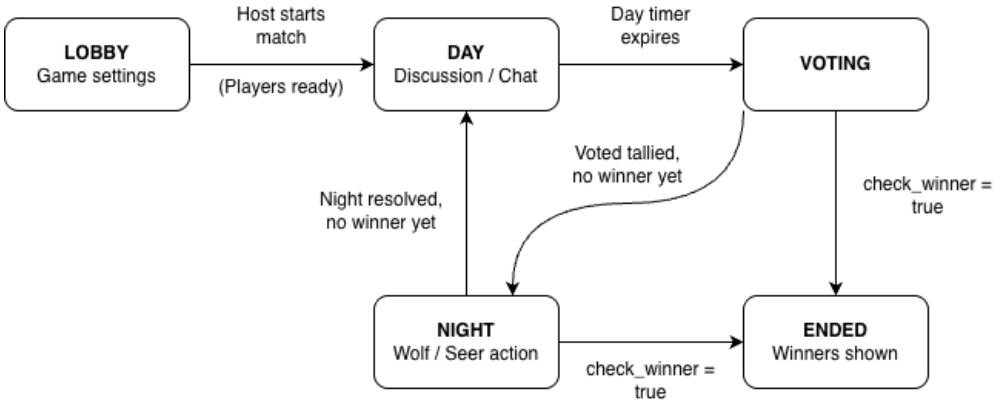

Figure 2: State machine of the match phases. Transitions are driven automatically by server-side timers and by `check_winner()`.

NFR3 – Fault recovery. The system shall recover quickly from temporary disconnections of Redis or of a backend instance.
Acceptance criterion: after a temporary disconnection, the backend re-establishes the connection and clients receive the correct state again without the match being recreated.

NFR4 – Responsiveness. Player actions shall be visible to other participants in real time or near real time.
Acceptance criterion: votes, phase changes, chat messages, and reconnections are propagated without requiring a manual page refresh.

NFR5 – Match data integrity. The system shall not lose the current match state during normal operation.
Acceptance criterion: after restarting a single backend replica, the match state remains recoverable from the shared data store.

NFR6 – Usability. The interface shall allow a non-technical user to join and play a match without needing to understand the system's internal details.
Acceptance criterion: a user can complete lobby creation/joining, the lobby phase, and the match using only a browser and a room code.

2.3 Implementation Requirements

IR1 – Python backend. The backend shall be implemented in Python using FastAPI and Socket.IO.
Acceptance criterion: the backend service exposes HTTP APIs and WebSocket/Socket.IO channels as specified by the design.

IR2 – Web frontend. The client shall be a Single Page Application built with Vue 3 and Vite.

Acceptance criterion: the application runs in the browser and communicates with the backend via REST and WebSocket.

IR3 – Shared state on Redis. The system shall use Redis as the shared store for lobby state, match state, votes, and timers.
Acceptance criterion: multiple backend replicas read and write the same state without relying on separate local data stores.

IR4 – Containerized deployment. The system shall be executable in containerized environments and support deployment via Docker Compose and Kubernetes.
Acceptance criterion: the full environment starts locally via Compose and can also be deployed on a Kubernetes cluster.

IR5 – Observability. The system shall expose metrics for monitoring purposes.
Acceptance criterion: the backend publishes metrics that can be queried by Prometheus and visualized in Grafana.

These constraints are consistent with the structure already adopted in the project repository: backend, frontend, Redis, reverse proxy, monitoring stack, and Kubernetes manifests.

2.4 Features Less Relevant or Not Central

- **Geographical distribution.** This is not the main motivation for distribution in this project. The system is not designed for users spread across the globe requiring geo-aware routing or replication, but rather to support multiplayer gameplay and backend replication for scalability and fault tolerance.
- **Heterogeneity beyond the standard web stack.** This is not a strong priority. The system's components are built with different technologies (Vue, FastAPI, Redis), but all communicate through standard protocols (HTTP, WebSocket), and no complex integration with heterogeneous third-party systems is required.
- **Strong security model.** Security is not the main focus of the project. The system does not process critical personal data or financial transactions; security concerns are essentially limited to enforcing correct game actions and host privileges.

3 Design

This chapter explains the strategies adopted to meet the requirements identified in the analysis, independently of the specific technologies used in the implementation.

3.1 Architecture

The system adopts a three-tier architecture with event-driven logic: presentation in a Vue Single Page Application on the client, application logic in a replicated FastAPI/Socket.IO backend, and shared state in Redis. The backend does not treat its in-memory representation as the authoritative source of truth; rather, it reconstructs domain state from Redis on demand and propagates events between replicas through a Redis Pub/Sub channel.

This style follows directly from NFR1 (state consistency across clients and replicas) and NFR2 (availability under replica failure). Because clients may connect to any backend instance behind the load balancer, no single process can be allowed to hold the only copy of a match's state: if it did, a client routed to a different replica would see a divergent or missing view of the game. Centralizing authoritative state in Redis, and using Pub/Sub purely as a notification mechanism rather than as a source of truth, allows the backend fleet to be horizontally scaled and to lose or gain replicas without breaking the consistency of an ongoing match. This directly satisfies IR3 (shared state on Redis) and IR4 (containerized deployment supporting multiple replicas), while keeping the architectural style independent of the specific deployment target.

3.2 Infrastructure (NON SO SE E' TUTTO GIUSTO)

The infrastructural components required by the design are: the browser clients, a static frontend server, a reverse proxy / ingress, a pool of backend replicas, a shared in-memory data store (Redis), and a monitoring stack (metrics exporter, Prometheus, Grafana). Concretely this means one frontend service, one logical backend service exposed through multiple replicas, one broker/state-store instance, one externally facing proxy, and two monitoring components, in addition to the client population.

The introduction of a reverse proxy / ingress is dictated by NFR2 and IR4: since the backend is replicated, something external to the application must be responsible for distributing connections across instances and for presenting a single entry point to clients. The introduction of Redis as a separate infrastructural component follows from IR3: a key-value store reachable by every replica is the only way to keep NFR1 satisfied without resorting to sticky sessions, which would reintroduce a single point of failure per session. The monitoring stack (IR5) is kept as a separate concern from the game logic, consistent with the requirement that observability must not affect the correctness of the state machine.

Regarding physical/network placement, the project does not require multi-datacenter or multi-continent distribution, consistently with Section 2.4 (geographical distribution is explicitly out of scope). In the Docker Compose deployment, all services run on the same host but are segmented into separate bridge networks so that, for instance, only the backend and Redis can reach each other and the frontend cannot talk to Redis directly. In the Kubernetes deployment, frontend, backend, and Redis are deployed in the same cluster and the same namespace; backend and Redis are exposed only as internal `ClusterIP` services, while external traffic enters exclusively through the NGINX

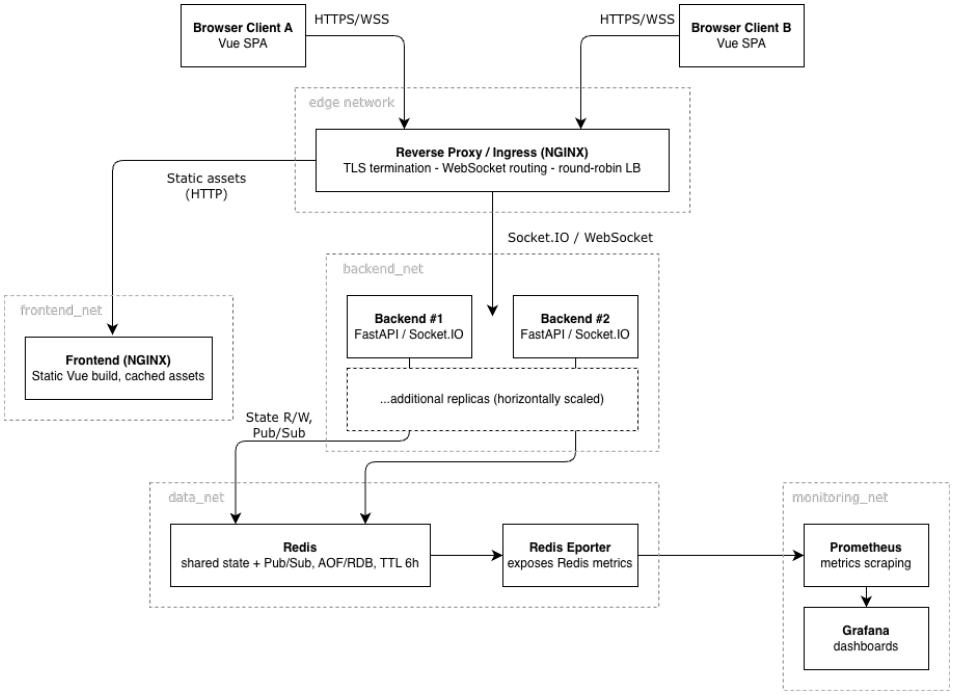

Figure 3: Component diagram

Ingress. When run locally via Minikube, all components effectively remain on the same node, but the manifests remain valid for a genuinely multi-node cluster, which is the deployment target implied by IR4.

Component discovery relies on internal DNS rather than hard-coded addresses, so that the same container images work unmodified across environments. In Docker Compose, service names (`frontend`, `backend`, `redis`) are resolved through the Compose-managed DNS; in Kubernetes, the cluster DNS resolves the corresponding Service names. The frontend receives the backend's externally reachable address at runtime through an environment variable (`WS_URL`), rather than at build time, so a single frontend image can be deployed against different backend endpoints. Load balancing across backend replicas is delegated to the proxy/ingress layer (detailed in Section 3.8), which is also how newly joining or reconnecting clients "find" an available backend instance without needing to know which physical replica currently serves a given room.

3.3 Modelling

The core domain entities are `Player`, `GameState`, `Role`, `Phase`, and `Winner`. At the level of application state this is complemented by the set of votes cast in the current phase, the active phase timer, the identifier of the host, the set of players who have signalled readiness in the lobby, and the final match result.

The mapping of these entities onto infrastructural components follows the single-

source-of-truth principle already introduced in Section 3.1: authoritative state lives in Redis, phase timers are scheduled locally by whichever backend replica currently owns them but are always validated against the deadline timestamp persisted in Redis, and the frontend keeps only a local, disposable representation of this state in a Pinia store and in session storage. This choice follows from the mapping suggested by NFR1 and NFR5 (match data integrity): since the frontend's copy is never authoritative, losing it (e.g. on a page reload) is not a data-loss event. The client can always resynchronize from Redis through the backend.

The domain events supported by the system model the lifecycle of a match: players joining or leaving, players signalling readiness, updates to lobby settings, role assignment, votes being cast, eliminations, the outcome of the seer's night action, the match being paused or resumed, and the match ending. These events are distinct from the messages actually exchanged over the wire, which include both commands sent from client to server and events broadcast from server to clients. Commands include `player_ready`, `update_settings`, `cast_vote`, `wolf_vote`, `seer_action`, `kick_player`, `play_again`, and `return_to_lobby`; events include `game_state_sync`, `player_joined`, `player_left`, `phase_changed`, `role_assigned`, `vote_update`, `game_paused`, `game_resumed`, and `game_ended`, among others. Separating commands (intents, which may be rejected) from events (facts, which have already happened) keeps the authorization logic of Section 3.9 straightforward: a command can fail server-side validation, while an event, once emitted, is simply a notification of a state change that has already been committed to Redis.

Overall, the state of the system comprehends: room and player identifiers, the current phase and its deadline, role assignments, the vote map for the day phase and the equivalent structure for the werewolves' night vote, the seer's investigation result, the set of ready players, the host identifier, and the final winner once the match concludes. Grouping this information under a room-scoped key in Redis (rather than, say, one global table) directly reflects the natural partitioning of the domain: no query ever needs to span multiple rooms at once, which will also justify the choice of a key-value model in Section 3.6.

3.4 Interaction

Components communicate through two channels: a WebSocket-based Socket.IO channel between each client and whichever backend replica accepted its connection, and a Redis Pub/Sub channel among all backend replicas. On connection, the client supplies a `client_id` and a `room_id` as part of the Socket.IO authentication payload; the backend checks that the identifier is not already active in the room, verifies that the current match phase still allows new participants to join, registers the socket in the room, and replies with a full state snapshot through a `game_state_sync` event. This request/response-like exchange at connection time is what allows a reconnecting or newly routed client to obtain a consistent view of the match regardless of which replica served the previous connection, directly supporting NFR1 and NFR3 (fault recovery).

During the lobby phase, the interaction pattern is asymmetric: the host may modify

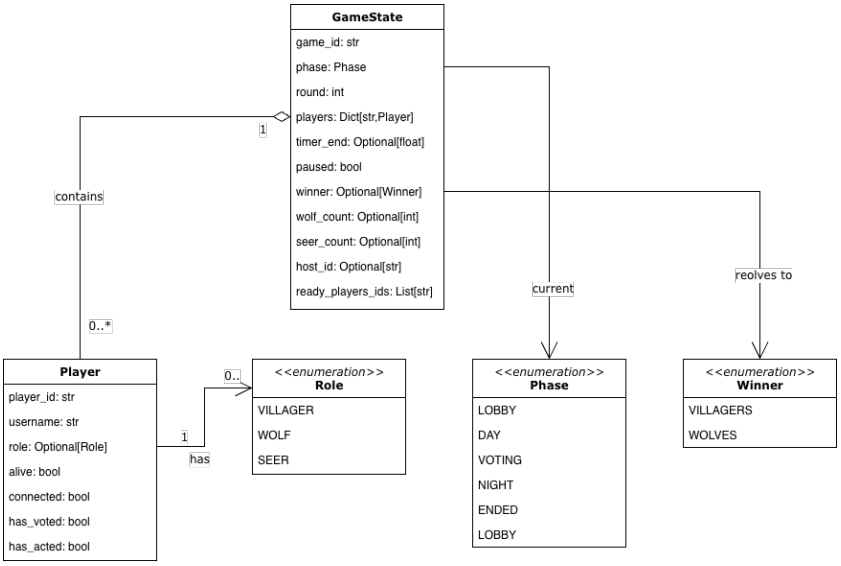

Figure 4: Class diagram

settings, start the match, or remove a player, while regular clients may only mark themselves as ready; during the match, players send either a vote or a night action, and the backend decides when a phase has collected enough input to advance. This keeps the client thin and prevents any client from unilaterally advancing the shared state, which is a prerequisite for the consistency guarantee of NFR1.

Cross-replica interaction follows a publish/relay pattern: the replica that processes a command publishes the resulting event on a Redis channel; every other replica subscribed to that channel receives it and re-emits it to its own locally connected clients. Private events (such as a player's assigned role or the seer's investigation outcome) are addressed to a specific `target_client_id` so that the relaying replica can deliver them to the correct socket even when the recipient is connected to a different instance than the one that generated the event. This pattern effectively turns the pool of stateless-with-respect-to-domain-state replicas into a single logical broadcast domain, which is what makes NFR1 achievable without a shared-memory backend process.

3.5 Behaviour

Individually, each backend replica behaves mostly as a stateless request handler with respect to game logic: no replica keeps a private, authoritative copy of any match. What each replica does retain locally is short-lived bookkeeping: the mapping from local socket identifiers to room/client pairs, the asyncio tasks backing phase timers and disconnection grace periods, and its own Pub/Sub subscriptions. This separation is intentional: it allows any replica to be killed and restarted without losing match data, since nothing irreplaceable lives only in that process's memory.

State updates are performed through Redis primitives chosen specifically for the con-

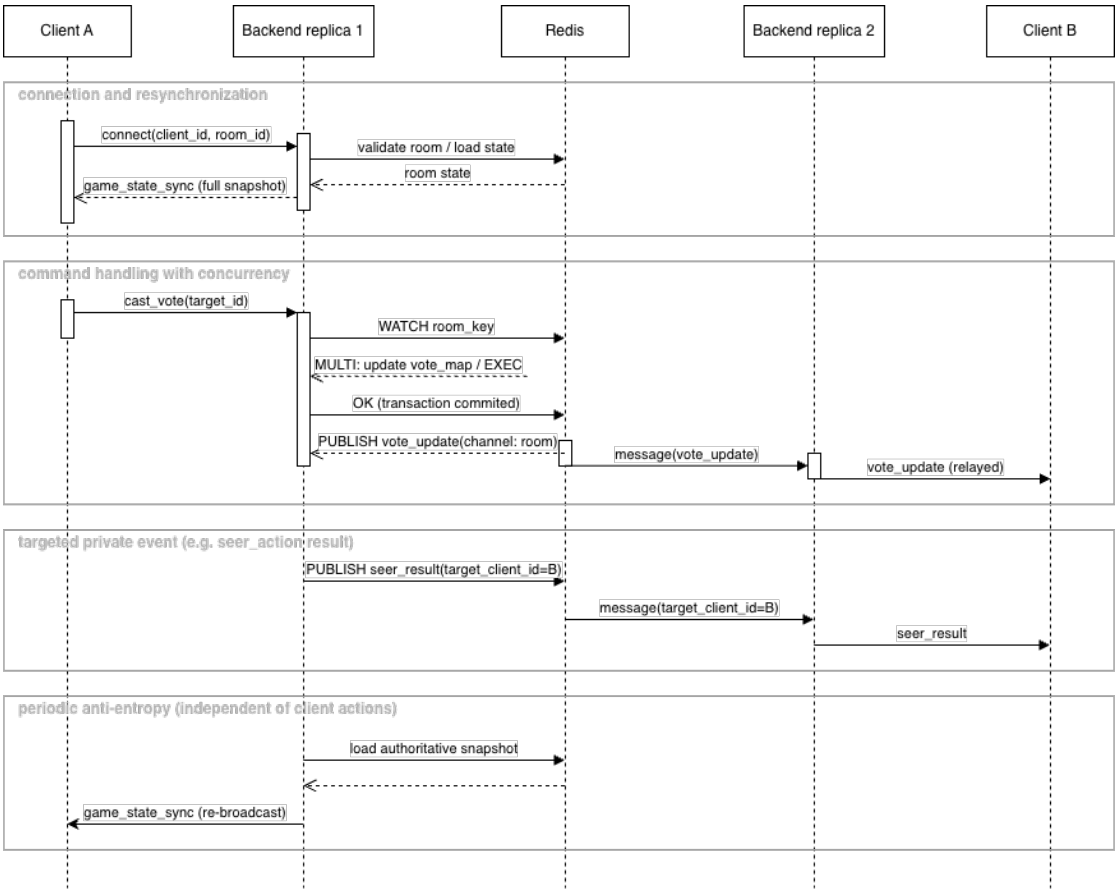

Figure 5: Sequence diagram

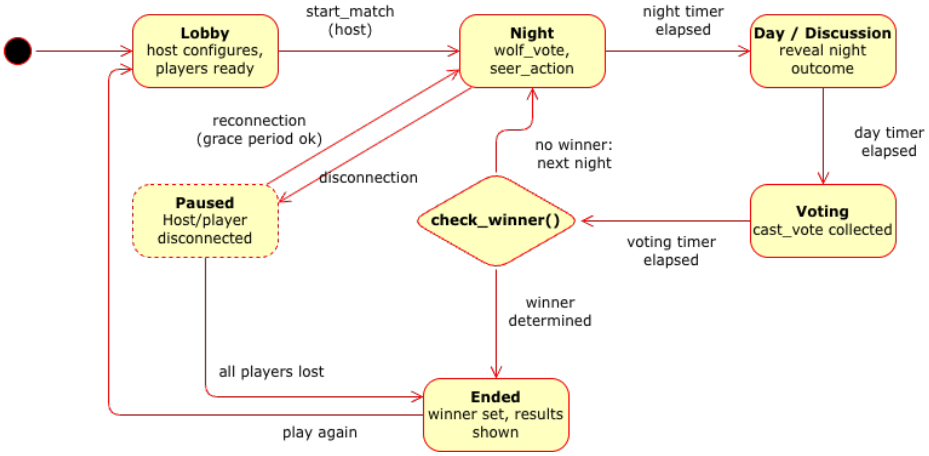

Figure 6: State diagram

currency guarantees they provide: `SET NX` is used to atomically create a room and elect its first host, avoiding a race where two clients both believe they created the same room; `WATCH/MULTI` transactions guard read-modify-write sequences (such as updating the vote map) against lost updates when two replicas act on the same room concurrently; pipelining is used to batch bulk updates without incurring multiple round trips; and a `NX`-with-TTL lock serializes phase-advancement so that only one replica actually transitions a room from one phase to the next, even if several replicas' timers fire close together.

The progression of a match is driven by two complementary mechanisms. Each replica schedules local timers for the phase it believes is active, and phase advancement is attempted when a timer fires; a periodic sweeper additionally scans for phases whose deadline has already elapsed, which recovers cases where the owning replica died before its timer fired. In parallel, an anti-entropy loop periodically re-broadcasts an authoritative snapshot of the match state to connected clients, which corrects for any Pub/Sub message that a client might have missed due to a transient disconnection. Together, these two mechanisms are the principal way the design satisfies NFR3 and NFR4 (responsiveness): the system does not rely solely on the reliable, in-order delivery of every single event, because reliable delivery cannot be assumed across process and network boundaries.

Disconnection handling follows a similar philosophy: if a player disconnects while a match is in progress, they are not removed immediately but are placed in a twenty-second grace period during which reconnection restores their previous state; only if that window elapses without a reconnection is the player eliminated. If the disconnecting player happens to be the host, a new host is elected among the remaining participants. This grace period is a direct response to NFR4 and to the fact that WebSocket connections over real networks are expected to drop transiently; treating every disconnection as final would make normal network jitter indistinguishable from a player intentionally leaving.

3.6 Data and Consistency Issues

The data that needs to be persisted comprises the full match state, the list of players and their attributes, the current votes and werewolves' votes, the seer's pending or resolved action, the timestamp marking the end of the current phase, and the set of currently active rooms; all of it is stored in Redis with a six-hour time-to-live, after which an abandoned room is automatically reclaimed. Storing this data in Redis rather than only in backend process memory is what makes NFR5 achievable: restarting a single replica must not destroy an ongoing match, and it does not, because the replica never held the only copy of that data to begin with.

The persistence model is key-value, using Redis hashes and sets rather than a relational schema. This choice follows from the access pattern implied by the domain model of Section 3.3: essentially every read and write is scoped to a single room, there are no cross-room joins or aggregate queries, and what the game needs are frequent small reads, small atomic updates, and fast propagation of state between replicas (exactly the operations Redis is optimized for, and a poor match for the overhead of a relational engine).

All queries against Redis are issued by the backend; the frontend never talks to the data store directly, which keeps every state mutation subject to the authorization checks discussed in Section 3.9. The most significant queries are: scanning the set of currently open lobbies, loading the full state of a room on (re)connection, reading the list of players in a room, reading and writing the vote map during a phase, and retrieving the deadline of the active timer. Because multiple replicas may read and write the same room concurrently (for instance, two players voting at nearly the same time through two different replicas) the design assumes concurrent reads and concurrent writes as the normal case, not an edge case, and addresses them with the `WATCH`/`MULTI` transactions, the phase-advancement lock, and Pub/Sub event deduplication already described in Section 3.5.

Certain pieces of state are, by construction, shared between components rather than owned by a single one: the room identifier, the client identifier, the host identifier, the set of ready player identifiers, the vote map, the phase deadline, and the room snapshot broadcast to clients. For each of these, the frontend only ever holds a disposable, locally cached copy, while the backend and Redis jointly hold the authoritative version.

3.7 Fault-Tolerance

The current deployment does not replicate Redis itself: it runs as a single instance backed by a persistent volume with AOF/RDB persistence enabled, so a Redis restart can recover its last persisted state, but Redis remains a single point of failure for the data tier. This is an accepted trade-off given the scope defined in Section 2.4 (the project's goal is backend replication for scalability and fault tolerance at the application tier, not a fully replicated storage layer) but the application code is nonetheless written defensively with respect to temporary Redis unavailability, since a transient network partition between a backend replica and Redis is a realistic failure mode even with a single Redis instance.

Concretely, fault tolerance at the communication layer relies on retries with exponential backoff for `PUBLISH` operations, automatic reconnection to the Redis broker, deduplication of events that a replica might otherwise re-process after reconnecting, and resubscription to the relevant Pub/Sub channels once a connection is restored. These mechanisms exist specifically to satisfy NFR3: the system must recover quickly from a temporary disconnection of Redis or of a backend instance, without requiring the match itself to be recreated.

At the client level, if a WebSocket connection drops, the frontend attempts reconnection and, once reconnected, rebuilds its view of the match entirely from the `game_state_sync` snapshot rather than from any locally cached delta. This is deliberate, since a locally reconstructed state could easily have drifted from the authoritative one during the outage. At the infrastructure level, if a backend replica crashes, Kubernetes restarts the corresponding pod, and clients previously connected to it are routed to a surviving replica by the ingress on their next reconnection attempt. Finally, error handling at the application level favours explicit rejection over silent recovery: invalid or out-of-order commands result in an explicit error response, or in the connection being refused outright, rather than being partially applied. This is what prevents a malformed or illegitimate action from corrupting the shared match state.

3.8 Availability

There is no separate application-level cache in the design; Redis itself serves as the single shared state layer, and the only caching present is ordinary HTTP caching of static frontend assets (JavaScript, CSS, images) served by the frontend's NGINX container with aggressive cache headers. A dedicated cache in front of Redis was judged unnecessary given the access pattern described in Section 3.6: state is small, room-scoped, and already served from an in-memory store, so introducing a second caching tier would add complexity and a further consistency concern without a clear performance benefit.

Load balancing is present at two points, consistent with IR4's requirement that the system support both Docker Compose and Kubernetes deployment. In the Compose setup, a reverse-proxy NGINX instance sits in front of both the frontend and the backend; in Kubernetes, the NGINX Ingress distributes WebSocket connections across backend replicas, effectively round-robin. This is what allows the backend to be scaled horizontally to satisfy NFR2, since new connections (and, on reconnection, previously served clients) can land on any healthy replica rather than being pinned to the one that first accepted them.

In the presence of a network partition, the system is designed to fail soft rather than to fail closed. A client that loses connectivity to its backend replica simply sees its WebSocket connection drop; on reconnection, it recovers a consistent view of the match from the shared source of truth in Redis rather than from any state it may have cached locally, satisfying NFR3. Symmetrically, if a backend replica temporarily loses its connection to Redis, it attempts reconnection rather than failing over to a degraded local state, and phase timers are protected by comparing the locally scheduled deadline against the deadline currently stored in Redis before acting, which prevents a replica that was par-

titioned from Redis for a period of time from advancing a phase twice or from acting on a deadline that has since been superseded. Finally, the Kubernetes deployment is configured for backend high availability independently of any specific partition scenario: it starts with two replicas, performs rolling updates without downtime, and uses readiness, liveness, and startup probes together with a `preStop` hook to drain in-flight connections before a pod is terminated, which together support NFR2 under both planned rollouts and unplanned failures.

3.9 Security

The system does not implement strong authentication in the sense of usernames and passwords, tokens, or JWTs. A player's identity is instead based on a `client_id` generated and stored client-side and passed as part of the Socket.IO connection handshake. This is a deliberate simplification consistent with Section 2.4, which explicitly places a strong security model outside the project's priorities: the system does not handle personal data beyond a nickname, nor financial transactions, so the cost of a full authentication subsystem was judged disproportionate to the actual risk surface.

Authorization, in contrast, is enforced at the application level and is not optional: only the host may change lobby settings, start the match, remove a player, or restart the room; only players who are still alive may vote or perform a night action; and only players holding the relevant role may invoke role-specific night actions such as the seer's investigation or the werewolves' vote. This access-rights model is coarse-grained by design: a single privileged role (host) versus regular players, further refined by liveness and role checks during the match, which matches the actual set of roles the game defines and avoids introducing an access-control abstraction more general than the domain requires.

Server-side validation also rejects requests that are inconsistent with the current phase of the match, independently of who issues them: a new client cannot join a match that has already started or already ended, and a duplicate nickname within the same room is rejected at connection time. This phase-aware validation is what keeps the authorization model of the previous paragraph meaningful and it directly supports the state-consistency goal of NFR1 by preventing illegitimate transitions from ever reaching the shared state in Redis.

No cryptographic scheme is applied at the application level: there is no message signing and no end-to-end encryption of the game payloads. Transport security is instead delegated to the deployment layer; the Docker Compose configuration already reserves port 443 for a future TLS-terminating proxy, but in the current codebase traffic between clients and backend travels as plain HTTP/WebSocket. This is consistent with the scope defined in Section 2.4: given that the payloads exchanged are gameplay events rather than sensitive data, the immediate priority was correctness of the distributed game logic rather than confidentiality of its traffic, with TLS termination left as an infrastructural addition rather than an application-level concern.

4 Implementation

4.1 Network protocols

Two application-level protocols are used, one per kind of interaction identified in Section 3.4. Plain `HTTP` backs the REST endpoints exposed by FastAPI, used for stateless, request/response interactions such as fetching the list of open lobbies; `Socket.IO` backs every realtime, bidirectional interaction between a client and a match: commands, events, and reconnection. The backend runs as an ASGI server exposing both protocols on the same process, while the frontend uses `socket.io-client` for the realtime channel and the standard `fetch` API for REST calls.

This split is a direct consequence of the interaction patterns of Section 3.4: REST is adequate for the handful of interactions that are genuinely request/response and do not need to push anything to the client afterwards, whereas the core of the game (state synchronization, phase changes, votes) is inherently event-driven and long-lived, which HTTP alone does not model well.

Between backend replicas, synchronization does not reuse Socket.IO or HTTP at all: it happens over Redis Pub/Sub, i.e. over the Redis wire protocol on top of TCP, which is also the same connection already used for state persistence. Reusing the existing Redis connection for both state and cross-replica notification avoids introducing a separate message broker purely for this purpose, which would have added an additional infrastructural component and an additional failure mode without a corresponding benefit at the scale this project targets.

4.2 In-transit data representation

All messages in transit are represented as `JSON`. Socket.IO event payloads are modelled in the backend as Python `dataclass` instances and explicitly converted to JSON-serializable dictionaries before being emitted; messages exchanged through Redis Pub/Sub follow the same convention, being serialized to JSON strings before publishing and parsed back on receipt. Redis itself stores game state as JSON strings together with native hash and set structures.

JSON was preferred over a binary format such as Protocol Buffers or MessagePack mainly because Socket.IO is built around JSON-friendly payloads on the JavaScript side, and because the message volume and size involved (small event payloads, a handful of times per second per room at most) do not justify the added complexity of schema compilation and binary encoding/decoding on both the Python backend and the Vue frontend. Using the same representation for the wire format and for the Redis-persisted state also means a payload can, in most cases, be stored and broadcast without an intermediate transformation step.

4.3 Database querying

No relational, SQL-based database is used. Persistence is delegated entirely to Redis, and all interaction with it is NoSQL in nature: key-value and hash-based operations

such as `GET`, `SETEX`, `SET NX EX`, `HGET`, `HGETALL`, `HSET`, `SADD`, and `SMEMBERS`, combined with `WATCH/MULTI/EXEC` transactions for read-modify-write sequences and a Lua script for atomically releasing the phase-advancement lock. The access pattern is room-scoped, read-heavy, and needs small atomic updates rather than relational joins, so a NoSQL key-value store maps onto it more directly than a SQL schema would.

Game state is treated as the authoritative shared source and is additionally given a six-hour time-to-live, so that abandoned or forgotten rooms are reclaimed automatically instead of accumulating indefinitely in memory. The TTL delegates garbage collection to Redis instead of requiring a separate scheduled task, which is consistent with keeping backend replicas as stateless as possible, as discussed in Section 3.5.

4.4 Authentication

The current implementation does not include strong authentication. Instead, the client generates (or retrieves, if already present) a `client_id`, stores it in `sessionStorage`, and sends it as part of the Socket.IO connection handshake; the backend treats this identifier as a session identity rather than as a cryptographic credential.

This is a deliberate implementation-level simplification of the trade-off already discussed at the design level in Section 3.9. It is adequate for the controlled, ephemeral environment the game runs in, but it would need to be replaced by a real authentication mechanism before the system could be exposed in a context where impersonating another player's `client_id` carried any real consequence.

4.5 Authorization

Authorization is implemented entirely server-side, through application rules that combine the requester's role, the current phase of the match, and per-player attributes such as liveness and connection status. At the lobby level, only the host may update lobby settings or start the match. During a match, gameplay actions are enabled only in the phase they belong to and only for the players entitled to perform them: `cast_vote` is accepted only during the `VOTING` phase, while `wolf_vote` and `seer_action` are accepted only during `NIGHT` and only from players holding the corresponding role, with additional checks ensuring the player is still alive and has not already performed the action in the current phase.

4.6 Technological details

On the backend, the implementation is built on `FastAPI` for the REST surface, `python-socketio` for the realtime layer, `Pydantic` for request/response validation and serialization, `Redis` (via its async Python client) for shared state and Pub/Sub, and a `Prometheus` client library for exposing application metrics.

On the frontend, the implementation uses `Vue 3` as the component framework, `Pinia` for client-side state management, `Vue Router` for navigation between lobby and game views, `socket.io-client` for the realtime connection, and `Vite` as the build tool and development server. Pinia's role here is worth making explicit given Section 3.3:

it holds only the client's disposable, locally cached view of the match, rebuilt from 'game_state_sync' whenever necessary, and is never treated by the rest of the frontend code as a source of truth in its own right.

At the infrastructure level, `NGINX` acts as the reverse proxy in front of frontend and backend and as the static file server for the built frontend bundle; `Docker Compose` orchestrates local multi-container deployment; `Kubernetes` is the target for a horizontally scaled, multi-replica deployment; and `Prometheus` together with `Grafana` provides metrics collection and visualization.

5 Validation

Validation combines three layers of automated testing: unit, integration, and end-to-end, with manual acceptance testing of the parts of the system that automated tests do not cover well.

5.1 Automatic Testing

5.1.1 Unit testing of individual components

Domain logic and the Redis access layer are unit-tested in isolation, using `fakeredis` in place of a real Redis instance so that these tests remain fast, deterministic, and free of external dependencies. On the backend, `backend/tests/test_game_logic.py` covers role assignment, winner computation, vote tallying, night-action resolution, and phase transitions; `backend/tests/test_lobby_logic.py` covers host election, ready-state handling, and lobby rules; `backend/tests/test_state_store.py` covers persistence and retrieval of state through the Redis access layer. On the frontend, Vitest tests under `frontend/src/__tests__/` cover the Pinia store, composables, and the mapping between incoming Socket.IO events and store mutations.

The rationale for this layer is to validate the correctness of functions that are pure or nearly pure (game rules, state serialization, UI-facing derived values) and to catch regressions on them independently of the network or of Redis. This is also where corner cases are exercised most directly, such as a player with no assigned role yet, an already-expired phase timer, a match with the minimum allowed number of players, or an empty initial lobby state. These tests are automated with `pytest` and `pytest-asyncio` on the backend and with `vitest` on the frontend; fixtures and mocks make it possible to run the full suite repeatedly without starting Redis or a browser. They are run with:

```
cd backend
pytest

cd frontend
npm install
npm run test:unit
```

Because this layer targets domain logic rather than distributed behaviour, it does not map to a single numbered requirement; instead, it underlies the correctness assumptions that every higher-level requirement (in particular NFR1, state consistency) depends on.

5.1.2 Integration testing among components

A second layer of tests verifies interaction between modules that cooperate on the same piece of state within a single replica, rather than each module in isolation. This includes `backend/tests/test_lobby_runtime.py`, `backend/tests/test_backend-init.py`, and `backend/tests/test_distributed_safety.py`. These tests check both that the runtime logic emits the expected events for a given sequence of commands, and that the Redis concurrency primitives introduced in Section 3.5 (atomic host creation, the phase-advancement lock, concurrent state patches, and bulk persistence) behave correctly under the conditions they were designed for.

The rationale here is that individually correct components can still misbehave once combined, and the places where this is most likely to happen are exactly the sensitive points of the distributed design: two replicas racing to create the same room, two timers racing to advance the same phase, two concurrent patches to the same room's vote map, and the propagation of state through Redis Pub/Sub. `backend/tests/test_distributed_safety.py` in particular targets this layer directly, covering atomic host creation, the phase-advancement lock, atomic state patching via a `state_version` field, active-room bookkeeping, and bulk pipeline updates. These tests are automated with the same `pytest`/`pytest-asyncio` setup as the unit-test layer, and are run selectively with:

```
cd backend
pytest backend/tests/test_lobby_runtime.py
pytest backend/tests/test_distributed_safety.py
```

5.1.3 End-to-end / system testing

The repository also includes standalone Python scripts that open genuine Socket.IO/WebSocket connections against a running backend, exercising the system as a whole rather than through direct function calls: `tests/test_multi_instance.py`, `tests/test_reconnection.py`, `tests/load_test.py`, and `tests/diag_loss.py`. Their rationale is precisely to cover what the two lower layers cannot: real client/server communication over the network, cross-instance propagation through Redis Pub/Sub, reconnection behaviour, event deduplication, and robustness under load (i.e. the behavioural guarantees promised by NFR1 through NFR4 and by the fault-tolerance mechanisms of Section 3.7, observed end-to-end rather than assumed from unit-level correctness).

`test_multi_instance.py` and `test_reconnection.py` support both a URL reached through the ingress/reverse proxy and direct access to individual pods via `kubectl port-forward`, which allows the same test logic to validate both an already-load-balanced deployment and a specific replica in isolation.

Layer	What it validates	Rationale	Requirement(s)
Unit	Domain rules, state serialization, UI derivations, in isolation via `fakeredis` / mocks	Fast, deterministic feedback on pure logic and corner cases	Underlies NFR1
Integration	Cooperating runtime modules and Redis concurrency primitives on a single replica	Individually-correct components can still misbehave once combined at sensitive points	Atomic host creation, phase lock, atomic patching, active-room bookkeeping, bulk updates
End-to-end	Real network communication, cross-replica propagation, reconnection, load	Only observable with a running system, not from function calls	NFR1–NFR4

Table 1: Summary of the automated testing layers.

The corner cases explicitly exercised at this layer include: loss or delay of Pub/Sub messages, a client reconnecting to a different backend instance than the one it was originally connected to, a duplicate-host race, both local (same-replica) and cross-instance broadcast, bursts of messages arriving in a short window, and disconnection followed by state recovery on reconnection. They are run as:

```
# System test distributed across multiple replicas
python tests/test_multi_instance.py --url http://localhost

# Reconnection and state consistency
python tests/test_reconnection.py --url http://localhost

# Message-loss diagnostics
python tests/diag_loss.py ws://game.local/ws

# Load test
python tests/load_test.py --url http://localhost --clients 500 --procs 4
```

Before running these more expensive tests, there are also some lightweight smoke checks meant to confirm that the backend, the WebSocket layer, and the reverse proxy are all reachable:

```
curl http://localhost/health
wscat -c "ws://localhost/ws/test_room?client_id=test123"
```

5.2 Acceptance test

Alongside automated testing, the project relies on manual acceptance testing focused on the parts of the system that are inherently tied to how a real user experiences the application through the browser. The scenarios exercised manually are the complete gameplay flow (creating and joining a lobby, host handling, toggling ready/not-ready, starting a match, progressing through phases, performing night actions, and viewing the final result) together with the behaviour of the interface when a player disconnects and later reconnects, real-time updates of player lists, timers, votes, and roles, and visual/behavioural consistency across multiple browsers and multiple tabs.

This testing was not automated for a specific reason: what it verifies is partly a visual and interactive quality: layout, role banners, the perceived responsiveness of the interface, and the consistency between backend state and what is actually rendered on screen, rather than a functional property that a script can assert on. Fully automating it would require a complete browser-based end-to-end layer, which was judged to be considerably more expensive to build and maintain than the combination of logic-level automated tests already described in Section 5.1 and targeted manual passes over the actual UI.

6 Deployment

6.1 Installing the system from scratch

6.1.1 Prerequisites

Running the project end to end requires `Docker Desktop 24.x` and `Docker Compose v2.x` in all cases; `kubectl 1.28+` and, for a local cluster, `minikube 1.32+` are additionally required for the Kubernetes deployment path. `Node.js 20` and `Python 3.12` are only needed if the frontend or backend are to be run or tested outside their containers; they are not required for a purely containerized deployment, since all application dependencies are already bundled into the corresponding images. These versions are not arbitrary but match the base images actually used in the project: the backend image is built from `python:3.12-slim`, the frontend image from `node:20-alpine`, and the infrastructure containers use `redis:7.2-alpine`, `prometheus:2.51.0`, and `grafana:10.4.0`.

6.1.2 Full local installation with Docker Compose

The recommended flow for a first installation is to clone the repository, ensure the Docker daemon is running, and then build and start the full stack, explicitly scaling the backend service to more than one replica so that the distributed behaviour described in Chapter 3 is actually exercised rather than degenerating to a single-instance deployment:

```
docker compose build
docker compose up -d --scale backend=2
```

The state of all containers can then be inspected with `docker compose ps`, and the backend's health endpoint can be checked directly:

```
docker compose ps
curl http://localhost/health
```

Once the stack is up, the system exposes: the web interface at `http://localhost`; the WebSocket endpoint at `ws://localhost/ws/<room_id>`; the REST API under `http://localhost/api/`; Prometheus at `http://localhost:9090`; and Grafana at `http://localhost:3001`.

The expected outcome of this installation is that every container reported by `docker compose ps` is either `running` or `healthy`; that the frontend is reachable from a browser at the address above; that `curl /health` returns an `ok` status; that the backend accepts WebSocket connections; and that both Prometheus and Grafana are reachable for monitoring.

6.1.3 Development-mode installation

For iterative development, the repository also supports a lighter setup in which only infrastructural services (Redis and the monitoring stack) run in containers, while the backend and frontend run directly on the host:

```
docker compose -f docker-compose-dev.yml up -d
cd backend && pip install -r requirements.txt && uvicorn main:app --reload --port 8000
cd frontend && npm install && npm run dev
```

This mode trades the full-stack, production-like guarantees of the Compose deployment (reverse proxy, multiple backend replicas, containerized frontend) for fast reload cycles during development, and is not intended as the final deployment target described in the rest of this chapter.

6.2 Environment variables and configuration

The project makes extensive use of environment variables and configuration files internally, but this does not translate into manual work for whoever installs the system: for a standard installation, following the instructions of Section 6.1 is sufficient, and no environment variable needs to be set by hand. Every variable the application depends on is already defined, with a working default value, directly inside the deployment manifests: `docker-compose.yml` and `docker-compose-dev.yml` for the Compose path, `k8s/configmap.yml` and the other `k8s/monitoring/*.yml` manifests for the Kubernetes path. Running `docker compose up` or `kubectl apply -f k8s/` is therefore enough on its own; there is no separate `.env` file to create or edit first, there is nothing left for an installer to fill in.

6.3 Containerization strategy

6.3.1 Docker Compose

`docker-compose.yml` defines the full local environment. It sets up separate networks for the frontend, backend, data, and monitoring tiers, consistent with the network segmentation already discussed in Section 3.2; uses NGINX as the reverse proxy in front of frontend and backend; allows the backend service to be scaled horizontally via `--scale backend=N`; attaches a persistent volume to Redis; attaches dedicated volumes to Prometheus and Grafana; includes a `redis_exporter` service for Redis metrics; defines a health check for every service; sets `restart: unless-stopped` throughout; and exposes ports only where external access is actually needed.

The repository distinguishes a development mode from the full deployment: `docker-compose-dev.yml` starts only the supporting infrastructural services for local development against a host-run backend and frontend, while `docker-compose.yml` starts the complete application stack described above.

The expected outcome of a Compose deployment is that the site is reachable at `http://localhost`; that the API responds under `http://localhost/api/`; that WebSocket traffic is correctly proxied by NGINX under `/socket.io/`; that Grafana and Prometheus are operational; and that the backend can be scaled to additional replicas without any change to the application code, only to the `--scale` argument.

6.3.2 Kubernetes

For a production-like deployment, the repository includes a complete set of Kubernetes manifests under `k8s/`.

The architectural choices encoded in these manifests follow directly from the availability and fault-tolerance goals: the backend starts with `replicas: 2` and is allowed to scale up to 8 replicas through a Horizontal Pod Autoscaler; the frontend is likewise replicated and stateless; Redis runs as a single replica backed by a persistent volume claim; Prometheus and Grafana each have their own dedicated persistent volume claims; rolling updates are configured with `maxUnavailable: 0` so that a deployment update never reduces the pool of available backend replicas below its current size; readiness, liveness, and startup probes govern how Kubernetes judges a pod's health at each stage of its lifecycle; a `preStop` hook on the backend drains in-flight connections before a pod is terminated; and two separate Ingress rules split traffic between the game itself and the monitoring stack, both served under the reference hostname `game.local`.

Deployment onto a cluster follows a fixed manifest order, so that each component's dependencies (namespace, configuration, data tier) are already in place before the component itself is applied:

```
kubectl apply -f k8s/namespace.yml
kubectl apply -f k8s/configmap.yml
kubectl apply -f k8s/redis/
kubectl apply -f k8s/backend/
```

```
kubectl apply -f k8s/frontend/
kubectl apply -f k8s/monitoring/redis-exporter.yml
kubectl apply -f k8s/monitoring/prometheus.yml
kubectl apply -f k8s/monitoring/grafana.yml
kubectl apply -f k8s/ingress.yml
```

or, equivalently, through the bundled deployment script:

```
chmod +x k8s/deploy.sh
./k8s/deploy.sh
```

The expected outcome is that all pods reach the `Running` state; that the backend and frontend Deployments report `Ready`; that the Ingress is reachable at `game.local`; that `http://game.local/health` returns `ok`; and that Grafana and Prometheus are reachable under `http://game.local/grafana` and `http://game.local/prometheus` respectively. The deployment can be verified with:

```
kubectl get pods -n game
kubectl get services -n game
kubectl get ingress -n game
kubectl get hpa -n game
curl http://game.local/health
```

7 User Guide

- how to use your software?
  - provide instructions
  - provide expected outcomes
  - provide screenshots if possible

8 Self-evaluation

- An individual section is required for each member of the group
- Each member must self-evaluate their work, listing the strengths and weaknesses of the product
- Each member must describe their role within the group as objectively as possible.

It should be noted that each student is only responsible for their own section.

9 Future works

Discuss possible future works, improvements, extensions, optimizations, etc.

Also discuss here any unimplemented feature, and how you would have implemented it.

References

[1] D. Adams. *The Hitchhiker's Guide to the Galaxy.* San Val, 1995.